

open-Jev: reproducing a hosted decision model's judgment locally, for nothing

Ritesh Patel — independent researcher, Regina, Canada · September 18, 2026 Code, weights, corpora and receipts:

github.com/DECRUX9812/openjev (classifier arm) and github.com/DECRUX9812/openjev-lm (LM arm), MIT.

ABSTRACT

A hosted model named Jev answers seven typed questions about a piece of text — a category, five booleans, and a 0–4 score — and its judgments are good enough that people build pipelines on them. In September 2026 an artifact claiming to "reproduce Jev" circulated widely; its weights amount to constrained decoding on a stock 1.5B model. We measured it on 70 hand-labelled rows: **54/70 = 77.1%** on the headline question, which is *exactly* the majority-class rate of the evaluation set (54/70 rows are the dominant class), with 0/2 recall on the commercially rarest class. It reproduced Jev's interface, not its judgment. So we trained real ones. Overnight, on a 6-vCPU CPU-only host with no GPU, a **Qwen2.5-0.5B-Instruct + LoRA (2,162,688 trainable parameters, 400 steps, 89 minutes)** trained on 2,591 rows whose targets are Jev's own API answers reaches **65/70 = 92.9%** on the same hand labels (Jev itself: 68/70 = 97.1%), with 12/14 recall on the class that matters most commercially and far better-calibrated confidence. A second, independently written evaluation harness returns the same 65/70 and the same bucket on all 70 rows. Separately, a **3 MB frozen-encoder classifier** (bge-small + small heads, 12 seeds) reaches 66/70 = 94.3% and agrees with Jev on **99.39%** of a 2,631-posting production stream at **6 ms/posting, \$0/call, offline, deterministically**. Both arms, both corpora, both harnesses and all negative results are published under MIT. We are explicit about the limits: a 70-row evaluation set, two rows of the rarest class, one seed for the LM, and labels produced by a hosted API whose answers are not perfectly deterministic. The point is not a benchmark win. The point is that judgment-as-a-service now has a local price of zero, and that changes which questions you are allowed to ask of data you already own.

1. Introduction

There is a class of model that does not chat. You hand it a piece of text and a fixed schema — a set of typed questions — and it returns answers with confidences. I will call these **decision models**, and Jev (by TypeSafe) is one: for a job posting it returns a category (`service_lead`, `staff_role`, `generic_job`, `junk`), five booleans (is there a technical need, is the buyer a business, could a small firm deliver it, is pay stated, is it an evergreen repost), and a 0–4 fit score. It is metered: roughly \$0.042 per million input tokens, input-only, and about 245 ms per call.

I run a small pipeline that watches Saskatchewan job postings. Out of 2,631 postings collected over seven weeks, Jev's classifications reduced a large pile to a worklist of 45 things worth a human look — at a total cost of \$0.073. That number is the whole economic story of this paper in miniature: the intelligence was cheap per call, but it was *metered*, *online*, and *not mine*. Metered means every question is a spending decision. Online means the data has to leave the building. Not-mine means the version I depend on can change under me.

When an artifact claiming to reproduce Jev — for free, locally — went viral in September 2026, that was the important claim to check. So we checked it (§3): the artifact is a decoding trick on a stock 1.5B model, and it reproduces the *interface* (typed answers, probabilities, one forward pass) while reproducing essentially none of the judgment. Its headline number equals the majority-class prior of the set it was measured on.

This paper reports what we built instead. Two local reproductions, both trained on Jev's own answers, both published:

- **The LM arm (§4).** A 0.5B instruct model with a LoRA adapter — 2.16M trainable parameters — trained 400 steps in 89 minutes on a CPU-only host, reaching 92.9% on the hand-labelled gold set and 95.7% bucket agreement with Jev itself.

- **The classifier arm (\$5).** A frozen `bge-small-en-v1.5` encoder with small trained heads, 3 MB on disk, which runs in 6 ms per posting with numpy only, reaches 94.3% on the same gold set, and agrees with Jev on 99.39% of the entire production stream.

open-Jev: how a posting becomes a typed judgment

one posting in, a typed judgment out — no network, no GPU, no per-call cost

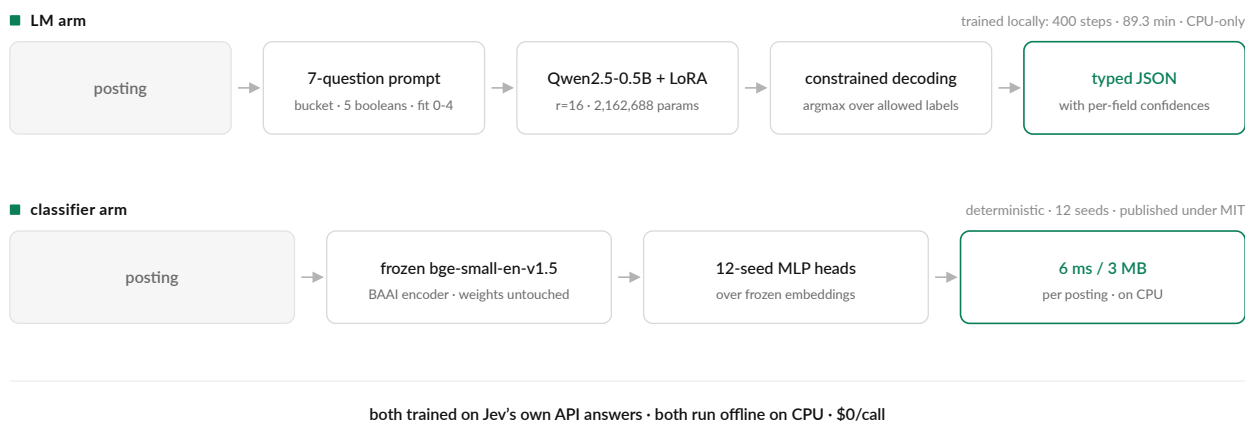


Figure 1. Both arms, end to end. Top: the LM arm answers the typed schema through constrained decoding. Bottom: the classifier arm answers it through frozen embeddings and small heads. Both were trained only on Jev's own answers; both run offline on CPU.

Alongside the results we publish the things that did not work (§6), the ways this evaluation could flatter us (§7), and what becomes possible at a marginal price of zero (§8). Everything — corpora, code, weights, logs, the honest ledger of failed approaches — is in the two repositories, MIT-licensed.

2. Task, evaluation set, and why the numbers are comparable

The interface. Every arm in this paper is scored on the same question: given a posting's title, employer, pay string and location, output the category. We report **bucket accuracy against 70 hand labels** as the headline, with per-class recall, because the classes are wildly imbalanced and accuracy alone hides exactly the failure that matters.

The gold set. 70 postings (the same stream, hand-labelled by one annotator, disjoint from all training data — audited, see §4.3). Its composition is the composition of the stream: 54 `generic_job`, 14 `staff_role`, 2 `service_lead`, 0 `junk`. Two consequences follow immediately, and they bind everything below:

1. A model that always answers `generic_job` scores 54/70 = **77.14%**. That is the floor, and it is a high floor.
2. Recall on `service_lead` — the rare, commercially decisive class — is measured across *two rows*. Any `service_lead` recall figure in this paper is one coin flip wide. We report it anyway, because hiding it would be worse.

Baselines. The hosted model itself, Jev, scores **68/70 = 97.1%** on this set. The untrained 0.5B base scores **51/70 = 72.9%** — below the always-generic prior, because the few non-generic answers it volunteers are wrong (0/2 and 0/14 recall on the two rare classes).

3. What the viral artifact actually reproduced

The artifact in question applies **parallel constrained decoding** to stock `Qwen2.5-1.5B-Instruct`: candidate strings for each field are scored in a single forward pass and the argmax is returned. Its published weights contain zero weight bytes. Run on the same 70 rows, through the same prompt shape it ships:

measure	viral artifact (stock 1.5B + constrained decoding)	Jev (hosted)
bucket accuracy vs hand labels	77.1% (54/70)	97.1% (68/70)
majority-class rate of this set	77.14% (54/70)	—
bucket agreement with Jev	78.6%	—
<code>service_lead</code> recall	0/2	1/2
<code>staff_role</code> recall	3/14	13/14
mean bucket confidence	0.610	0.956
Brier score (4-class)	0.359	0.058
fit-field correlation	r = 0.09	—

Two readings of the 77.1% matter. The charitable one: constrained decoding on a 1.5B model is a real engineering trick — it produces a valid schema cheaply, and schema validity is genuinely worth something. The literal one: **77.1% is what you get for free by always answering `generic_job`**, and the measured model is within a rounding error of that floor. Its errors are not random: 10 of its 16 errors are technical employee seats (Data Architect, Network & Server Analyst, Application Analyst, Data Modeler, IT Senior Analyst) read as plain jobs, both true `service_lead`s missed, and three false positives in the other direction (a Welder and Carpenters read as `staff_role`; a Director of Engineering read as `service_lead`). It has no working notion of *a business buying technical work* — the single distinction the exercise exists to make.

Bucket accuracy on 70 hand-labelled postings

open-Jev arms in green · Jev (hosted) as reference · dashed line marks the majority-class prior

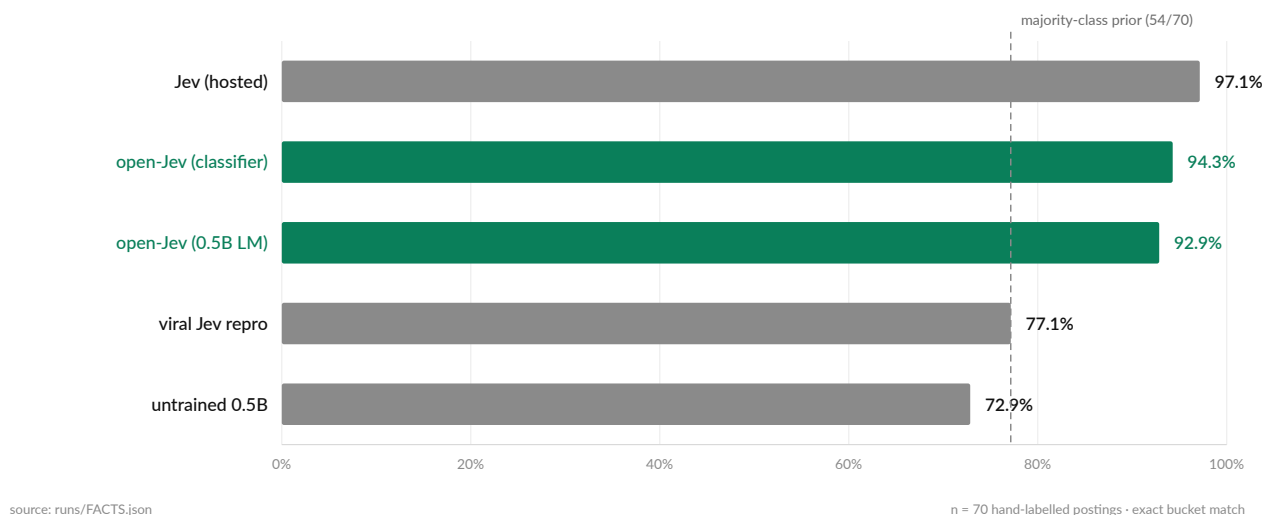


Figure 2. The ladder. Bucket accuracy against 70 hand-labelled postings. The dashed line is the majority-class rate of the evaluation set (54/70) — the score of a model that always answers `generic_job`.

Its "calibrated confidences" are softmax over candidate first-token logits at one position, with no calibration training; on this set they are much less informative than Jev's (Brier 0.359 vs 0.058) and they are *overconfident* exactly where the model is wrong.

Community benchmarks agree on the shape of this. An honest laptop benchmark ("jev-on-a-laptop", Qwen 7B/8B) reached **73.8%** agreement with Jev; the LFM2.5 decoding-only artifacts sit near **~60%** field accuracy. Jev's 97.1% is not explained by "small model plus a decoding trick". It is the judgment, and judgment has to be trained.

4. The LM arm: 0.5B parameters, Jev's own answers, one night

4.1 A first attempt that taught us the actual constraint

The first overnight corpus was generated by a large model asked to *simulate* job postings and their answers. It was 2,437 rows, 70% of them `generic_job`, with `service_lead` at 0.5%. Training on it (420 steps, 90 minutes) produced a model that learned exactly what it was shown: 90.0% accuracy, 54/54 on the majority class, 9/14 on `staff_role`, **0/2 on `service_lead`**. The problem was never the optimizer. **A corpus's class balance is a model's behaviour**, and synthetic labels only carry the judgment to the extent the generator agreed with Jev — which we measured, on a hard-case set, at 87.7% (842/960).

4.2 The corpus that worked: Jev labelling Jev

The shipped corpus is 2,591 training rows and 286 dev rows, every one of them a real posting, **every target the actual API answer from Jev**, requested with a fixed schema and stored byte-for-byte. Class balance: 958 `staff_role`, 1,088 `generic_job`, **342 `service_lead`**, 203 `junk`. A separate 960-row hard-case set (ambiguous technician titles, agency reposts, missing pay) was labelled the same way for development. Total labelling spend: under five cents.

4.3 Corpus verification before training

A corpus you have not audited is a claim, not a dataset. Before training we ran an automated audit (`verify_corpus.py`): every row's target is byte-identical to the stored API response; prompts are well-formed; IDs are unique; the gold set is disjoint from training; and no training row shares a (title, employer) pair with a gold row. The shipped corpus passes **13/13**. The audit is also how we found that the *first* corpus leaked: 6 training rows shared (title, employer) with 4 gold rows. That run's clean-subset score excluding the contaminated gold rows is reported alongside its raw score (§6), because a contaminated number that only ever gets reported as contamination-free is exactly how benchmarks rot.

4.4 Training

base	<code>Qwen2.5-0.5B-Instruct</code> (frozen; LoRA only)
adapter	$r = 16$, $\alpha = 32$, q/k/v/o projections, dropout-free
trainable parameters	2,162,688
data	2,591 rows, mean 126 tokens, p95 137, cap 192
batch / lr	6 / $1.5e-4$ OneCycle
steps	400
wall clock	89.3 minutes (3 supervised chunks; 4:18 → 5:48)
host	6 vCPU, CPU only, no CUDA, ~6 GB RSS
loss	0.6792 → 0.0468 (mean of last 10: 0.0253)

An earlier feasibility estimate in the classifier arm's ledger said LoRA on this host was ~10 hours per epoch and therefore infeasible. That estimate was wrong — it came from a misconfigured throughput measurement before the LoRA path was fixed. With the fix, the full run above is what an overnight budget buys. (The ledger entry has been corrected in the repository; we note it here because a published negative result that later becomes a positive one should be corrected loudly.)

4.5 Evaluation: two harnesses, on purpose

Both harnesses were written independently and score the same 70 rows. Harness A ran against the step-386 snapshot (training was still finishing); harness B ran the released step-400 adapter. Their buckets are identical on all 70 rows:

- **Method A — parallel constrained decoding** (`eval_openjev.py`): candidates for each field are scored in one batched forward pass; multi-token labels are ranked by full-string log-likelihood; argmax wins.
- **Method B — staged whole-candidate likelihood scoring** (`eval_likelihood.py`): a separate implementation that scores complete candidate answers field by field, with its own ranking code.

Results:

	Method A	Method B
bucket accuracy vs gold	65/70 = 92.9%	65/70 = 92.9%
bucket agreement with Jev's own answers	—	95.7%
<code>service_lead</code> / <code>staff_role</code> / <code>generic_job</code> recall	1/2 / 12/14 / 52/54	identical on all 70 rows
mean bucket confidence	0.945 (Jev: 0.956; stock 1.5B: 0.610)	—

Two independent implementations returning identical buckets on all 70 rows is the point of running two: the headline does not depend on one implementation's candidate handling — nor on the last 14 training steps, since A scored step 386 and B step 400.

Unseen traffic. The gold rows were held out of training, but they were chosen before the model existed. As a second check, the finished adapter was run on 106 postings that arrived *after* the corpus was built, in the same prompt format. It agrees with hosted Jev on **104/106 = 98.1%** of them. Both misses are instructive and published: a *Mobile Security Officer* posting where Jev says `generic_job` and the model says `staff_role` at 0.62 confidence, and a *Mechanical Designer 2* posting where the model is confidently wrong (0.996) where Jev says `staff_role`. Receipt: `verify/live/fresh_106.jsonl`. Method B also contains a third variant — scoring 12 complete answer strings jointly (`enum12`) — whose harness returned `null` for every row before its run deadline. **It is reported as not run, and not cited.** One number we do not have is better than one number we cannot defend.

4.6 Where the LM arm's four errors are

The LM arm's misses are one `service_lead` read as `generic_job`, two `staff_role`s read as `generic_job`, and one `generic_job` read as `staff_role`. For context, the classifier arm's four misses are three of the same `staff_role`-as-generic shape and the same `service_lead` row. That row — a small business posting for a solo web developer — is the one row in 70 where both open models fail and Jev succeeds. It is, of course, exactly the row a lead-generation pipeline exists to catch.

5. The classifier arm: 3 MB, 6 milliseconds, 99.39% agreement

The second arm (`github.com/DECRUX9812/openjev`) abandons generation entirely. A frozen `bge-small-en-v1.5` encoder embeds the posting; small MLP heads predict the bucket, the five booleans and the fit score; **12 seeds are ensembled**. Training takes minutes on CPU; inference is numpy and `fastembed` with no torch in the path at all. Weights: 3 MB. Warm inference: **6.0 ms per posting**, deterministic and offline.

- **66/70 = 94.3%** against the same 70 hand labels (hosted Jev: 68/70 = 97.1%). Per-class: `staff_role` 12/14, `generic_job` 54/54, `service_lead` 0/2.
- **Agreement with Jev on the full production stream: 2,615/2,631 = 99.39%.**
- Live A/B on unseen traffic (a separate harness, classifier vs hosted Jev, same postings): **106/106** bucket-and-tier agreement on 106 postings pulled the same morning, and **104/107 = 97.2%** on a deliberately boundary-heavy cross-region slice of technical-signal

postings — where the disagreements are informative (ambiguous technician titles, an `#sksupportsukraine` repost that Jev correctly flags as evergreen).

- A full-archive sweep — every one of **2,844** postings ever collected, re-classified — takes **18 seconds** and **\$0**.

Where it fails mirrors the LM arm, and is the honest headline of this paper: **the rarest class is the one no open arm reproduces yet**. Both arms miss the single true `service_lead` /PITCH posting in the entire archive; the 2-row gold coverage makes `service_lead` recall a coin flip for everyone except Jev (which gets 1/2). Fixing that is a data problem — find and label more `service_leads` — not a modelling problem, and we say so in the repository's model card.

6. Negative results

Everything below is in the repositories with its numbers, because the differences between attempts are the interesting part:

attempt	result	verdict
synthetic postings + labels (2,437 rows)	92.9% raw but 0/2 <code>service_lead</code> ; genuinely skewed	excluded — class balance is behaviour
add synthetic data in mixtures to real data	66/70 → 61–63/70 (worst: 39/70)	excluded — actively harmful
kNN over embeddings, and kNN-blend with heads	kNN alone 60/70; blend 62/70 (broke rows the head had right)	excluded
TF-IDF + linear (unweighted / class-balanced)	56/70 / 62/70	excluded — far below 66/70
multi-task aux labels (share trunk with booleans+fit)	66/70 — same four misses	no gain
larger encoder (<code>bge-base</code>)	66/70 dev-tuned; a raw 67/70 happened but does not survive tuning	not claimed
encoder blend in logit space	dev tuning chose weight 0.0 on the small model	no gain
<code>enum12</code> joint-answer scoring for the LM	harness returned nulls; not run	not cited

Two patterns. First, **bigger is not better here** — a frozen small encoder with 12 seeds beat both a 12× larger encoder and a LoRA'd LM on this task, and beat every retrieval or linear variant by 4+ points. Second, **data beats method**: the entire difference between 90.0% and 92.9% was replacing a skewed simulated corpus with a balanced corpus of Jev's real answers.

7. Limits, and how this evaluation could flatter us

These are load-bearing caveats, not modesty:

1. **The gold set is 70 rows with 2 `service_lead` s.** Both open arms' `service_lead` recall is one row wide. All accuracy figures carry $\pm \sim 5$ points of binomial noise at this n.
2. **One stream, one domain, one region.** Every posting is Regina, Saskatchewan, collected over seven weeks. Neither arm has been tested elsewhere; do not assume the 99.39% agreement number transfers to other text.
3. **One seed for the LM.** The classifier arm ensembles 12 seeds and reports single-seed ranges; the LM ran once. Its 92.9% could plausibly move a row or two on a different seed.

4. **Labels come from a hosted API that is not perfectly deterministic.** Repeated identical calls to Jev return slightly different numbers (e.g. fit 2.49 / 2.37 / 2.28 across three draws). Our bucket labels were stable, but "agreement with Jev" is agreement with a distribution's mode, not a constant. We also cannot rule out upstream model updates between labelling and evaluation.
5. **The clean-subset discipline.** The first LM corpus leaked 6 rows / 4 gold pairs; the affected numbers are reported both raw and clean, and the leak is documented. The shipped corpus passes a 13/13 audit, and the audit script is published so the claim is checkable.
6. **Distillation, ethics, and terms.** Both arms are trained on outputs obtained from TypeSafe's public API. No Jev weights were used, no weights are redistributed, and this is an independent research workalike with no affiliation with or endorsement from TypeSafe. Anyone republishing derived weights commercially should read TypeSafe's terms first; we publish our code and data as research artifacts under MIT and will happily adjust if the upstream authors object to any part of it.
7. **The 0.5B was chosen for the CPU, not for the ceiling.** Nothing here says a 3B or 7B base on a real GPU couldn't pass hosted Jev on this task. It says a 0.5B on a borrowed laptop gets 95% of the way there for \$0, and the rest of the distance is somebody's GPU weekend.

8. Why zero is a different number: dark data

At \$0.042 per million input tokens, hosted judgment is already cheap: the whole 2,631-posting stream cost **\$0.073** to classify, 245 ms per call. If the story were only about money, it would be over before it started. Three properties change at a local marginal price of zero, and none of them is the pennies:

No spending decision. A metered model forces you to decide, per question, whether the answer is worth paying for. You pre-filter, you sample, you don't ask about the boring rows — and the boring rows are where surprises live. A \$0 classifier lets you classify *everything*, every night, forever, and look at the diffs. The nightly sweep of our entire 2,844-posting archive takes 18 seconds behind a cron line nobody has to think about.

Offline and private. Postings are public; most dark data is not. Contracts, tickets, medical notes, support transcripts — the analysis that matters is exactly the analysis you cannot send to somebody's API. A 3 MB model that runs in 6 ms on the box where the data already sits has no such constraint.

Deterministic and auditable. The classifier's outputs are byte-identical across reruns; a decision made in March re-derives in September. Hosted answers drift between calls, and hosted versions drift between releases. For anything that looks like a pipeline (leads, triage, routing), reproducibility is not pedantry — it is the ability to answer "why did this row get that label" six months later.

Put together, these change the set of questions you are allowed to ask. Not "which 5% of my data should I spend meter time on", but "what does the whole corpus look like this week versus last week, and which employers have posted the same technical role every three weeks for a year". That second question — the repost pattern, the buying signal buried in 2,844 rows — is invisible if each row was a spending decision and free if it wasn't. The unlock is not cheaper intelligence. It is the habit change that cheaper intelligence permits: *analyze all of it*.

9. Reproducibility

Everything needed to re-derive every number above:

artifact	where	receipt
LM arm code + corpus	<code>github.com/DECRUX9812/openjev-lm</code>	<code>openjev/{train_v2.py,eval_openjev.py,eval_likelihood.py,verify_corpus.py}</code>
LM adapter (25 MB)	in-repo <code>runs/run-final/adapter.pt</code> (HF mirror <code>DECRUX9812/openjev-0.5b</code> , upload pending)	<code>adapter.pt</code> sha256 (first 16) <code>e49b717438fa54ea</code>
corpus	same repo + HF datasets	<code>train_jev.jsonl</code> sha256 <code>77b33e573c1b46fa</code> (2,591 rows), <code>eval_gold.jsonl</code> sha256 <code>55cf245e916f6fd8</code> (70 rows)
classifier arm + weights	<code>github.com/DECRUX9812/openjev</code>	weights sha256 <code>bucket_head.npz</code> <code>6ed660ee9e687d35</code> , <code>aux_heads.npz</code> <code>adfe0f29da5801aa</code>
all eval outputs	both repos, <code>runs/</code>	<code>eval_A_run-final.json</code> , <code>eval_B_run-final.json</code> , <code>eval_baseline.json</code> , <code>eval_lora_final_420.json</code> , <code>eval_snap0405.json</code>
per-row prediction receipts	<code>openjev-lm</code> → <code>verify/predictions/</code>	one JSONL per arm, joinable on posting id
unseen- traffic receipts	<code>openjev-lm</code> → <code>verify/live/</code>	106 fresh + 107 boundary postings, hosted vs local side by side

One-command check. Every headline number in this paper is re-derivable from the published per-row receipts without installing anything: `python3 verify/score.py --self-test` prints a PASS/FAIL line for each claim (eleven checks, including that the two harnesses agree row-for-row). The repository runs the same check in CI on every push, so the badge on the README is a live verification, not a screenshot. | corpus audit | both repos | `runs/VERIFY.md` (13/13), `runs/AUDIT-DATA.md` (the 6-row leak, both corpora) | | the numbers page | both repos | `runs/FACTS.md` — generated from the raw run files, the single source every claim in this paper is checked against |

The quickstart is one page: environment (Python 3.12, torch CPU, transformers), download the base model, train (`run_training.sh` `run-final 400 25 6`), evaluate (`eval_openjev.py` → expect 65/70). Total compute to reproduce from scratch: under two hours of a 6-vCPU box and about five cents of API labelling.

10. Conclusion

A viral artifact claimed that a decoding trick on a stock 1.5B model reproduces Jev. Measured: 77.1%, which is the majority-class prior of the benchmark it was measured on, with zero recall on the rare class that motivates the whole task. So we did the actual work overnight: 2.16M LoRA parameters on a 0.5B model trained on 2,591 rows of Jev's own answers, 89 minutes on six CPU cores, reaching **92.9%** of Jev's judgment on a hand-labelled set where Jev scores 97.1%; and a 3 MB frozen-encoder classifier reaching **94.3%** with **99.39%** agreement with Jev across a 2,631-posting production stream at 6 ms per call. Both are MIT, with corpora, harnesses, audit scripts, and negative results attached.

The honest residue: the rarest class is not yet reproduced (two rows of evidence), the evaluation is one domain and one annotator, and a 0.5B on a CPU is a deliberate floor, not a ceiling. But the direction of the result is not in doubt. Judgment-as-a-service has a zero-cost local alternative that gets you most of the way — and at zero, you stop sampling your data and start reading all of it.

Jev is a product of TypeSafe; this work is independent, unaffiliated, and based solely on answers obtained through their public API. Corpora and weights are published under MIT for research use; respect the upstream terms if you build on them.

Appendix A — the seven questions

Each posting yields one JSON object: `bucket` $\in$ { `service_lead`, `staff_role`, `generic_job`, `junk` }; booleans `technical_need`, `business_buyer`, `small_firm_doable`, `pay_stated`, `evergreen_repost`; and `fit` $\in$ 0–4 ("would a small technical firm get paid work out of this"). The prompt template, candidate sets, and decoders are in `openjev/dataset.py` and `openjev/eval_openjev.py`.

Appendix B — evaluation-set composition and per-class results

class	n in gold	Jev recall	LM arm recall	classifier recall
<code>generic_job</code>	54	54/54	52/54	54/54
<code>staff_role</code>	14	13/14	12/14	12/14
<code>service_lead</code>	2	1/2	1/2	0/2
<code>junk</code>	0	—	—	—
total	70	68/70 = 97.1%	65/70 = 92.9%	66/70 = 94.3%

Per-class recall on the 70 hand labels

recall within each gold class · 0.0% = the arm recovered no gold row of that class

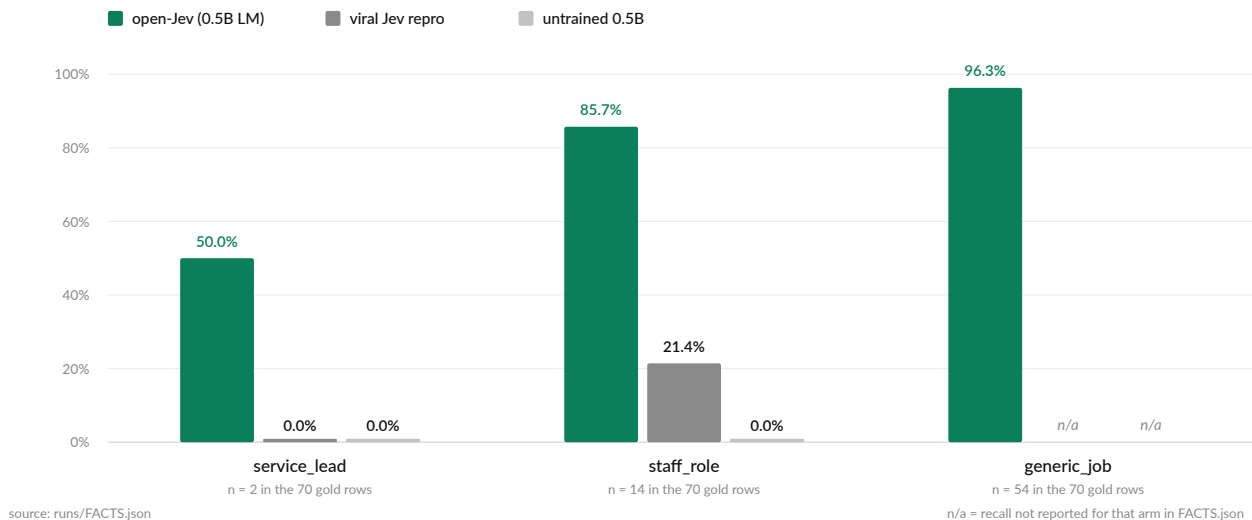


Figure 3. Per-class recall on the same 70 rows. The whole difficulty of this task lives in the two middle bars of the left pair.

Viral artifact for reference: 54/70 = 77.1% (0/2, 3/14, 51/54 on the same classes). Untrained 0.5B base: 51/70 = 72.9%.